MBS-fast: Physical Optics for Ice Crystals

Implementation Manual

2026-05-28

Contents

1 Overview

MBS-fast computes light scattering by non-spherical particles using the Physical Optics (PO) method. It traces geometric optics rays through the particle (reflections, refractions), then applies Kirchhoff diffraction to each output beam to obtain the far-field Mueller matrix.

This manual describes the current code, including CPU, MPI/OpenMP, CUDA, FFT angular interpolation, automatic averaging modes, cutoffs, checkpointing, and the integral characteristics printed in the logs. Formula names match the variables in `src/handler/HandlerPOTotal.cpp`, `src/scattering/TracerPOTotal.cpp`, and `src/cuda/GpuDiffraction.cu`.

2 Quick Start

```
# Simple CPU run: hex column, Sobol orientations, adaptive theta grid
cpu/bin/mbs_po_mpi --po --sobol 1024 --auto_tgrid 0.05 \
    -p 1 10 10 -w 0.532 --ri 1.31 0 -n 12 \
    --grid 0 180 48 180 --close

# GPU production-style file-particle run
gpu/bin/mbs_po_gpu_float_fast --po --pf Afine30.dat --k_eq 58.81 \
    --ri 1.6 0.002 -w 1.064 -n 14 \
    --oldauto 2 --grid 0 180 600 180 \
    --threads 12 --gpu --fft --checkpoint --close
```

3 CLI Reference

All flags are grouped by function. Required flags have no default; optional flags are marked with their defaults.

3.1 Particle Definition

3.1.1 `-p TYPE HEIGHT DIAMETER [EXTRA]`

Type	Shape	Parameters
1	Hexagonal column	H = height, D = diameter
2	Bullet	H, D (cap angle 62°, auto)
3	Bullet rosette	H, D, [cap height]
4	Droxtal	arg3 = sup parameter
10	Concave hexagonal	H, D, concavity depth
12	Hexagonal aggregate	H, D, number of elements

Symmetry: auto-detected. Hex column: $\beta_{\text{sym}} = \pi/2$, $\gamma_{\text{sym}} = \pi/3 \rightarrow 12\times$ reduction.

3.1.2 `-pf FILENAME`

Load particle geometry from file. The geometry is loaded, optionally resized by `-rs` or `-k_eq`, and then written to `particle_for_check.dat` for verification.

3.1.3 `-rs SIZE`

Resize particle loaded with `-pf` to maximal dimension SIZE (μm).

3.1.4 `-forced_convex` / `-forced_nonconvex`

Force convex or non-convex tracing algorithm.

3.2 Physical Parameters

3.2.1 `-w WAVELENGTH_UM`

Wavelength in micrometers. Examples: `-w 0.532` (green), `-w 1.064` (NIR).

3.2.2 `-ri N_REAL N_IMAG`

Complex refractive index $m = n_{\text{re}} + i\, n_{\text{im}}$. In the current code absorption accounting is enabled automatically when $n_{\text{im}} \neq 0$:

$$\texttt{isAbs} = \texttt{-abs} \ \vee \ (n_{\text{im}} \neq 0).$$

For $n_{\text{im}} = 0$, physical absorption is reported as zero even if the diagnostic angular integral of M_{11} does not exactly match the optical theorem.

3.2.3 `-abs`

Enable absorption. Requires `-w`. Uses `DiffractInclineAbs`.

3.2.4 `-n MAX_REFLECTIONS`

Value	Use case	Notes
5–6	Forward scattering, C_{sca}	$< 1\%$ error
12	General purpose	$< 5\%$ backscatter error
20	Backscattering	$< 3\%$ error
25	Reference calculations	

3.3 Method

- `-po` — Physical Optics (Kirchhoff diffraction). **Required for PO.**
- `-go` — Geometrical Optics (no diffraction).
- `-incoh` — Incoherent summation ($|J|^2$ per beam, then sum).
- `-karczewski` — Karczewski polarization matrix (experimental, same M_{11}).
- `-jones` — Output Jones matrices to file.
- `-all` — Compute all trajectory groups.

3.4 Orientations

3.4.1 `-sobol N` (recommended)

N Sobol quasi-random orientations (power of 2). Auto-uses particle symmetry.
Convergence: $O((\log N)^2/N)$, much faster than Monte Carlo $O(1/\sqrt{N})$.

x	C_{sca} ($< 2\%$)	$M_{11}(180°)$ ($< 5\%$)
10–50	128	512
50–200	256	1024
200–1000	512	4096
> 1000	1024	8192+

3.4.2 `-adaptive EPS`

Auto-determine orientations. Starts at 256, doubles until C_{sca} converges to relative accuracy EPS.

3.4.3 `-sym B G`

Override particle symmetry: $\beta \in [0, \pi/B]$, $\gamma \in [0, 2\pi/G]$.
Example: `-sym 6 2` $\rightarrow \beta \in [0, 30°]$, $\gamma \in [0, 180°]$ ($12\times$ reduction).

3.4.4 Other orientation modes

- `-random N_BETA N_GAMMA` — Uniform grid.
- `-montecarlo N` — Random orientations.
- `-fixed BETA GAMMA` — Single orientation (degrees).
- `-orientfile FILE` — Read (β, γ) pairs (radians) from file.
- `-b MIN MAX`, `-g MIN MAX` — Override ranges (degrees).

3.5 Scattering Grid

3.5.1 `-grid T1 T2 N_PHI N_THETA`

Uniform grid. **Use** $N_\phi \geq 48$; $N_\phi = 6$ causes 50–200% error.

3.5.2 `-auto_tgrid EPS` (recommended)

Auto non-uniform θ grid with 5 adaptive zones:

Zone	Range	Step	Purpose
Fine	0 to 10×peak	peak/20	Forward peak
Transition	to 20°	geometric ×1.3	Smooth bridging
Medium	to 120°	0.5° (x<100) or 1°	Side scattering
Side/back	to 175°	1°	Backscattering
Near-backscatter	to 180°	0.25°	LDR

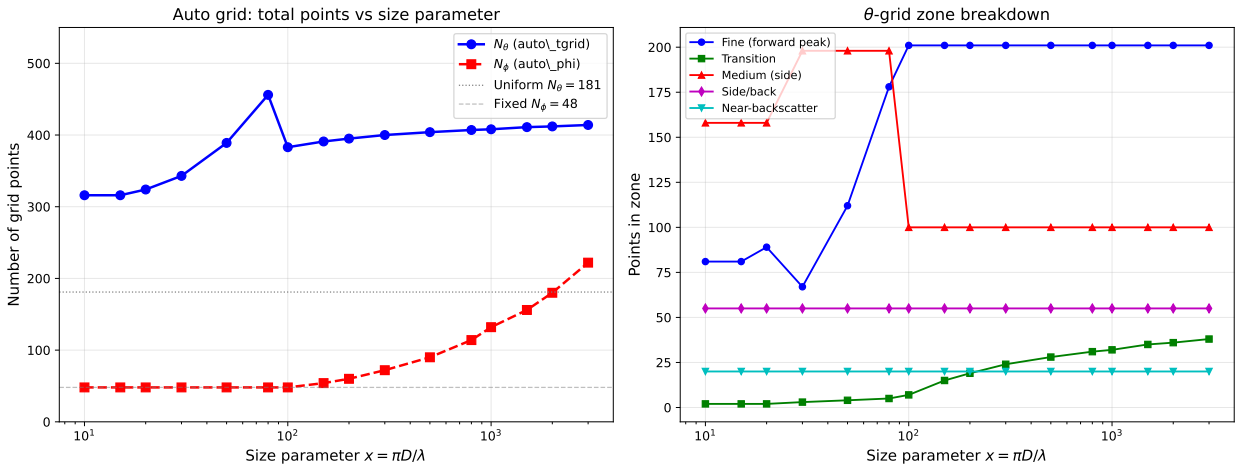

Figure 1: Left: total N_θ and N_ϕ vs size parameter x. Right: zone breakdown of θ-grid points.

3.5.3 `-auto_phi`

Auto-select N_ϕ based on the current size. The exact formula is selected in `main.cpp`; when `-mirror_gamma` or `-fft` is active the final value is rounded upward to a multiple of 6 by `MirrorSafePhiCount()`.

Recommended: use both `-auto_tgrid -auto_phi` together.

3.5.4 `-tgrid FILENAME`

Custom θ grid from file (degrees, one per line).

3.6 Multi-Size

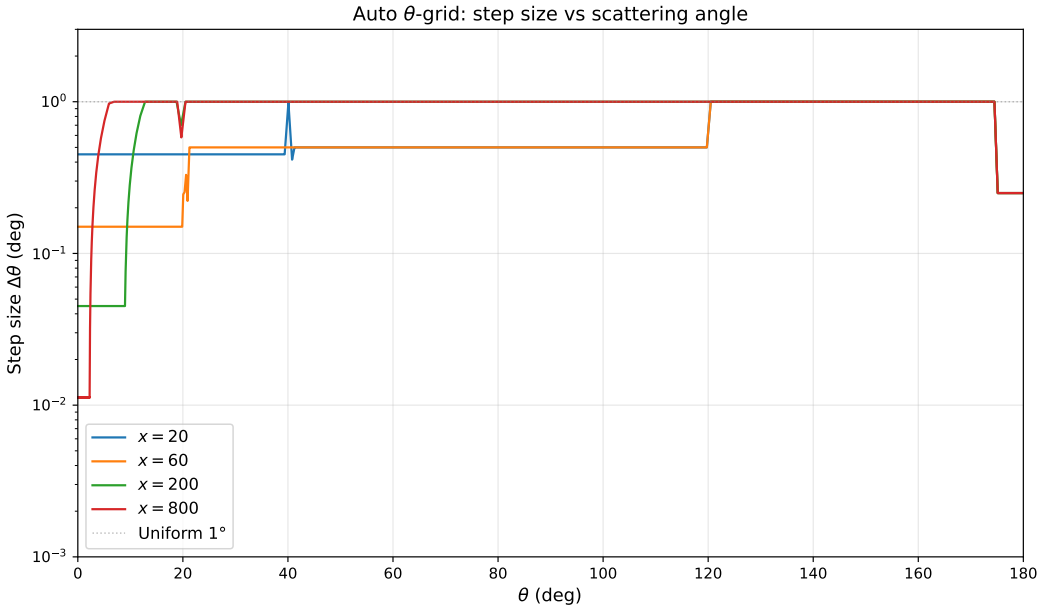

Figure 2: Step size $\Delta\theta$ vs scattering angle for different x. Small x has coarser steps (broad peak); large x needs fine steps near θ=0.

```
--multigrid Dmin Dmax N       # logarithmic Dmax scan
--multikeq Kmin Kmax N        # logarithmic k_eq scan
--multikeq_list file.txt      # exact k_eq values, one per line
```

With `-oldauto` the largest size is used as the shared reference for tracing. Beam geometry is cached at that reference size and diffraction is recomputed for each requested size. `-multigrid_parallel` instead starts independent child processes.

3.7 Performance Options

- `-beam_cutoff EPS` — Skip negligible beams.
- `-shadow_off` — Disable shadow beam (debug).
- `-r RATIO` — Small beam restriction ratio (default 100).

3.8 Output

- `-o DIRNAME` — Output folder (default `M`).
- `-close` — Exit after calculation (**required for scripts**).
- `-log SECONDS` — Progress output interval.
- `-tr FILE` — Trajectory groups from file.
- `-gr` — Per-group output.

4 Output Files and Console Log

4.1 Output files

All output is written to the directory specified by `-o` (default `M`).

4.1.1 `DIRNAME.dat` — Mueller matrix

Space-separated, 18 columns, one header line + one row per θ bin.

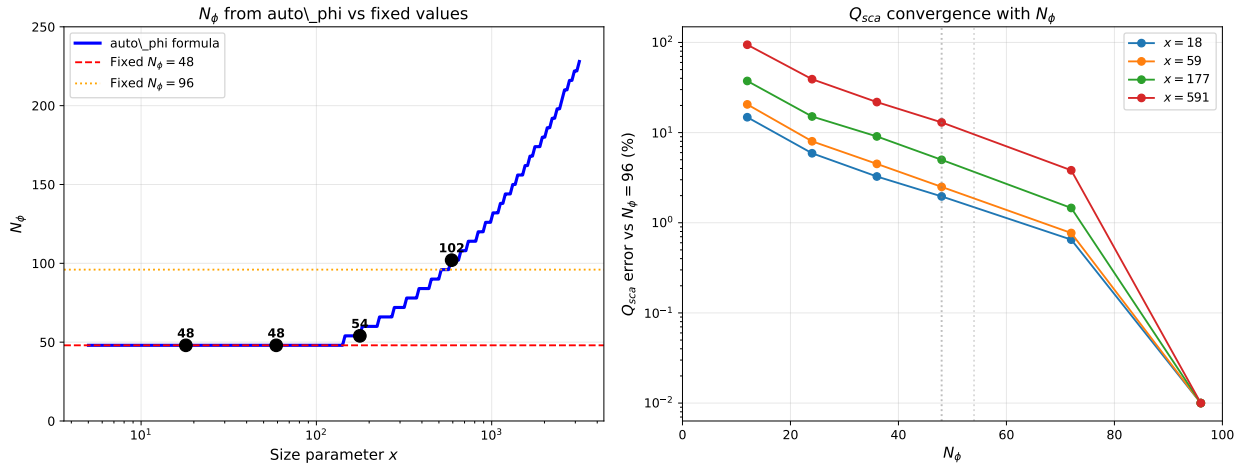

Figure 3: Left: N_ϕ from auto_phi (blue) vs fixed values. Right: Q_{sca} error vs N_ϕ for different sizes—larger x needs more ϕ bins.

```
ScAngle  2pi*dcos  M11  M12  M13  M14  M21  M22  M23  M24
                   M31  M32  M33  M34  M41  M42  M43  M44
```

- `ScAngle` — scattering angle θ in degrees
- `2pi*dcos` — solid angle weight: $2\pi\,\Delta\cos\theta$. Integration: $C_{\text{sca}} = \sum_j M_{11}(\theta_j) \times (2\pi\,\Delta\cos\theta_j)$
- `M11` ... `M44` — 16 Mueller matrix elements (phi-averaged). For randomly-oriented particles: $M_{13} = M_{14} = M_{23} = M_{24} = M_{31} = M_{32} = M_{41} = M_{42} \approx 0$.

Useful derived quantities:

$$Q_{\text{sca}} = C_{\text{sca}}/A_{\text{proj}}$$

$$g = \frac{\sum M_{11}\cos\theta \cdot 2\pi\, d\cos\theta}{\sum M_{11} \cdot 2\pi\, d\cos\theta} \qquad \text{(asymmetry parameter)}$$

$$-M_{12}/M_{11} = \text{degree of linear polarization}$$

4.1.2 `DIRNAME\out.txt` — run summary

Contains particle parameters, refractive index, method, start/end time, total number of orientations. Written after computation completes.

```
Particle:  Refractive index: 1.310000
  Area:429.903838
Number of secondary reflections: 6
Wavelength (um): 0.532000
Method: Physical optics, Sobol quasi-random
Start of calc.: Sun Mar 22 11:22:44 2026
End of calc.  : Sun Mar 22 11:22:49 2026
Total time  : 4s
Total number of body orientations: 128
```

4.1.3 `DIRNAME_jones.dat` — Jones matrices (with `-jones`)

Written when `-jones` flag is used (fixed orientation only). 10 columns:

```
# theta(deg) phi(deg) J00_re J00_im J01_re J01_im J10_re J10_im J11_re J11_im
```

One row per (θ, ϕ) grid point. Complex Jones matrix $\mathbf{J}$ relates incident to scattered field: $\begin{pmatrix} E_\parallel \\ E_\perp \end{pmatrix}_{\text{sca}} = \mathbf{J} \begin{pmatrix} E_\parallel \\ E_\perp \end{pmatrix}_{\text{inc}}$.

4.1.4 `particle_for_check.dat` — particle geometry

Dumped by the program for verification. Contains vertex coordinates and facet definitions of the particle as used in the calculation.

4.1.5 `beam_log.dat` — beam diagnostics

Written for the first orientation (debug). Columns:

```
# beam_num  nActs  area  opticalPath  |Jones|  cx  cy  cz  nVertices
```

`nActs` = number of internal interactions, `cx,cy,cz` = beam exit direction. The shadow beam has `nActs` = large number and `cz` = -1.

4.2 Console output (stdout)

The program writes progress and results to stdout.

4.2.1 Auto mode output

```
Auto theta grid: x=83.5, peak_width=2.16 deg, 464 points
Auto phi: x=83.5, N_phi=48
Adaptive mode: target relative change = 0.01
  N=64  (+64 new), M11(180)=7.09, dM11=100%, Csca=92.9, dCsca=100%
  N=128 (+128 new), M11(180)=6.69, dM11=5.6%, Csca=93.0, dCsca=0.12%
  N=256 (+256 new), M11(180)=6.64, dM11=0.8%, Csca=92.9, dCsca=0.08%
Converged at N=256 (both C_sca and M11)
```

Convergence messages:

- `Converged (both C_sca and M11)` — best: both criteria met
- `Converged (C_sca ok, M11 within X%)` — relaxed: M11 within $5 \times \varepsilon$
- `Converged (C_sca stable, M11 still X%)` — C_sca $< \varepsilon/10$ after 4 iterations
- `WARNING: Max orientations reached` — did not converge, result approximate

4.2.2 Sobol mode output

```
Sobol: 1024 orientations, beta_sym=30 deg, gamma_sym=180 deg
Phase 1 (tracing + preprocessing): 0.46 s
Phase 2 (diffraction, OpenMP): 13.25 s
Total: 13.71 s
```

4.2.3 Scattering efficiency (always printed)

```
===== SCATTERING EFFICIENCY: full =====
A_proj (incoming energy) = ...
Outcoming energy = ...
C_abs_GO = A_proj - outcoming energy = ...
C_ext = C_ext_OT (optical theorem forward amplitude) = ...
C_sca = ...
C_abs = ...
Q_sca = C_sca / A_proj = ...
Q_abs = C_abs / A_proj = ...
Q_ext = C_ext / A_proj = ...
EFFICIENCY_SUMMARY Qext=... Cext=... Qabs=... Cabs=...
```

The exact formulas and the non-absorbing special case are described in Section 5.4. If the diagnostic $Q_{\text{sca,integral}} > 2.5$, the code prints a warning because the angular integral of a narrow M_{11} peak is not reliable on a coarse theta grid.

4.3 Console output (stderr)

4.3.1 Memory diagnostic

```
Memory: 25219 MB available, chunk=4096 orientations (1 chunks)
Adaptive: max orientations = 32768 (25219 MB available)
```

4.3.2 Timing breakdown (at program exit)

```
===== HandlerPO TIMING BREAKDOWN =====
HandleBeams calls: 448
HandleBeams total:    69.255 s
  Beam loop total:    66.039 s
  AddToMueller:       2.606 s
  Unaccounted:        63.433 s
=======================================
```

"Unaccounted" = the optimized inline loop (batched sincos + edge loop + rotate_jones). Not instrumented to avoid timing overhead.

5 Current Implementation Reference

This section is the exact operational model used by the current code. It is intended to answer "what does this flag actually do" rather than describe an idealized method.

5.1 Particle size, equivalent radius, and `-k_eq`

For every particle the code computes volume V, surface area S, maximal dimension $D_{\max}$, equivalent-volume radius r_{eq}, and equivalent size parameter k_{eq}:

$$r_{\mathrm{eq}} = \left(\frac{3V}{4\pi}\right)^{1/3}, \qquad k_{\mathrm{eq}} = \frac{2\pi r_{\mathrm{eq}}}{\lambda}.$$

`-rs SIZE` rescales a file particle so that $D_{\max} =$ `SIZE`. `-k_eq K` rescales it so that $k_{\mathrm{eq}} = K$. The scale ratio is

$$a = \frac{K\lambda/(2\pi)}{r_{\mathrm{eq,current}}}, \qquad D_{\max,new} = aD_{\max,current}.$$

The log prints both `r_eq` and `k_eq`; this is the same definition used in ADDA-style comparisons.

5.2 Mueller output and azimuth averaging

For each output scattering angle θ_j, the code stores a full $N_\varphi \times N_\theta$ Mueller accumulator $M(p, j)$. The written one-dimensional matrix is an azimuth average. Away from the poles it applies the rotation matrix `Lp`:

$$L_p(\varphi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos 2\varphi & \sin 2\varphi & 0 \\ 0 & -\sin 2\varphi & \cos 2\varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

$$\bar{M}(\theta_j) = \frac{1}{N_\varphi} \sum_{p=0}^{N_\varphi - 1} M(p, j) L_p(\varphi_p).$$

At $\theta = 0$ and $\theta = \pi$ the code first averages without L_p and then projects onto the pole-symmetric Mueller form using `ApplyForwardPoleSymmetry()` or `ApplyBackwardPoleSymmetry()`.

5.3 Solid-angle weights

The second column in every `.dat` file is exactly the value returned by `ScatteringRange::Compute2PiDcos(j)`. For a uniform theta grid with step $\Delta\theta$:

$$w_j = 2\pi\,\Delta\cos_j\,.$$

For pole rows:

$$w_0 = w_N = 2\pi\left(1 - \cos\frac{\Delta\theta}{2}\right).$$

For interior rows:

$$w_j = 2\pi\left[\cos\left((j-\tfrac{1}{2})\Delta\theta\right) - \cos\left((j+\tfrac{1}{2})\Delta\theta\right)\right].$$

For a non-uniform `-tgrid`/`-auto_tgrid` grid, bin boundaries are midpoints between neighboring theta values:

$$w_j = 2\pi\left(\cos\theta_{j,-} - \cos\theta_{j,+}\right).$$

5.4 Integral characteristics

The following quantities are printed by `HandlerPOTotal::WriteMatricesToFile` and `WriteAveragedRowsFile`. They are intentionally separated because they answer different questions.

Projected area / incoming energy.

$$A_{\mathrm{proj}} = \sum_{\text{orientations}} A_{\mathrm{illuminated}}(\Omega_i)\,w_i.$$

This is printed as `A_proj (incoming energy)` and is the denominator for all efficiencies:

$$Q_x = \frac{C_x}{A_{\mathrm{proj}}}.$$

Angular scattering integral.

$$C_{\mathrm{sca,integral}} = \sum_j \bar{M}_{11}(\theta_j)\,w_j.$$

This is a numerical angular integral of the written M_{11} curve. It is very sensitive to the theta grid near a narrow forward peak. On a coarse grid it can be much larger or smaller than the optical-theorem value; the code then prints it as a diagnostic rather than silently using it as a physical truth.

Geometrical-optics energy balance. If absorption accounting is enabled, the code also accumulates outgoing beam energy from traced beams:

$$C_{\mathrm{abs,GO}} = A_{\mathrm{proj}} - E_{\mathrm{out}}.$$

If absorption is not enabled, the code sets $C_{\mathrm{abs,GO}} = 0$. This is printed as `C_abs_GO`; it is a diagnostic of the ray/beam energy balance and is not used to overwrite the optical-theorem extinction.

Optical-theorem extinction. For each prepared orientation the code evaluates the forward diffraction amplitude and accumulates

$$C_{\mathrm{ext,OT}} = \lambda\,\mathrm{Im}(f_{00} + f_{11})\,w_i,$$

where w_i is the orientation weight stored as `PreparedOrientation::sinZenith`. The global printed value is the sum over all orientations. The implementation is `HandlerPO::ComputeForwardExtinctionOt()`.

Reported cross sections. The final reported values are:

$$C_{\text{ext,legacy}} = C_{\text{sca,integral}} + C_{\text{abs,GO}}.$$

If optical-theorem extinction is available:

$$C_{\text{ext}} = C_{\text{ext,OT}}.$$

If OT is not available:

$$C_{\text{ext}} = C_{\text{ext,legacy}}.$$

For absorbing particles ($\operatorname{Im} m \neq 0$):

$$C_{\text{sca}} = C_{\text{sca,integral}}, \qquad C_{\text{abs}} = C_{\text{ext,OT}} - C_{\text{sca,integral}}.$$

For non-absorbing particles ($\operatorname{Im} m = 0$) and available OT:

$$C_{\text{abs}} = 0, \qquad C_{\text{sca}} = C_{\text{ext,OT}}.$$

The angular integral remains in the log as `Csca_integral`. If $|C_{\text{sca,integral}} - C_{\text{ext,OT}}|/|C_{\text{ext,OT}}| > 1\%$, the code prints a mismatch warning. This is expected on too-coarse theta grids.

5.5 Physical optics diffraction per beam

For each geometric beam the code stores a Jones matrix J, polygon area, edge data, local polarization bases, optical path, and projected length. The far-field Jones contribution is a product of:

$$J_{\text{sca}}(\theta, \varphi) = R(\theta, \varphi)\, F_{\text{Kirchhoff}}(\theta, \varphi)\, e^{i\Phi(\theta,\varphi)}\, J_{\text{beam}}.$$

The hot loop evaluates the polygon Kirchhoff integral through precomputed edge slopes. If both phase coefficients A and B are near zero, the analytic limit is used:

$$F_{\text{Kirchhoff}} = \pm \frac{i}{\lambda} A_{\text{beam}},$$

with the current default sign chosen by the forward-direction bugfix. The `-legacy_sign` flag restores the older sign convention.

The Jones-to-Mueller conversion in `AddToMuellerLocal()` uses the code's explicit component formulas. For example:

$$M_{11} = \frac{1}{2}\left(|J_{00}|^2 + |J_{10}|^2 + |J_{01}|^2 + |J_{11}|^2\right).$$

Default mode sums beams coherently within one orientation and then converts to Mueller. `-incoh` converts each beam to Mueller first and sums intensities. `-coh_orient` is a legacy mode that coherently sums Jones amplitudes across all orientations before converting; it is not the default physical orientation average.

5.6 Orientation weights and averaging modes

`-fixed BETA GAMMA.` Runs one body orientation. Angles are degrees.

`-random Nb Ng.` Uses a regular beta/gamma product grid over the particle symmetry domain. The current oldauto/random path uses beta midpoints by default:

$$\beta_i = \beta_{\min} + (i + \tfrac{1}{2})\Delta\beta,$$

unless environment variable `MBS_OLDAUTO_BETA_MIDPOINT=0` is set. Gamma points are

$$\gamma_j = \gamma_{\min} + j\Delta\gamma,$$

with an optional deterministic stagger in oldauto/random. The beta weight is computed by `CalcCsBeta()` from the cosine interval, and gamma is normalized by the number of gamma samples and the beta symmetry factor.

`-oldauto DIV`. This is the production regular-grid mode used for strict ADDA comparisons. It derives orientation density from a diffraction angular step:

$$\Delta\theta_{\text{diff}} = 0.69\,\frac{\lambda}{D_{\max}}\,\frac{180}{\pi} \quad \text{degrees.}$$

The orientation step is

$$\Delta\theta_{\text{orient}} = \frac{\Delta\theta_{\text{diff}}}{\texttt{-ring\_points}}, \qquad \texttt{-ring\_points default} = 3.$$

For particle symmetry ranges β_{sym} and γ_{sym}:

$$N_{\beta,\text{full}} = \left\lceil \frac{\beta_{\text{sym}}[\text{deg}]}{\Delta\theta_{\text{orient}}} \right\rceil, \quad N_{\gamma,\text{full}} = \left\lceil \frac{\gamma_{\text{range}}[\text{deg}]}{\Delta\theta_{\text{orient}}} \right\rceil.$$

$$N_\beta = \max\left(3, \left\lceil \frac{N_{\beta,\text{full}}}{\texttt{DIV}} \right\rceil\right), \quad N_\gamma = \max\left(3, \left\lceil \frac{N_{\gamma,\text{full}}}{\texttt{DIV}} \right\rceil\right).$$

`-oldauto 2` is stricter than `-oldauto 4`; `-oldauto 1` is the reference-level regular grid.

`-sobol N`, `-sobol_seed N S`. Uses a Sobol low-discrepancy sequence on the symmetry-reduced orientation domain. `-sobol` uses seed 42. `-sobol_seed` uses an explicit nested Owen scramble seed. Output weights are equal after mapping to the symmetry domain.

`-sobol_ring Nb Ng`. Uses a Sobol-like beta distribution combined with a uniform shifted gamma ring. This is useful when gamma periodicity aliases specular events.

`-hammersley`, `-lattice`, `-lattice_z`. Alternative low-discrepancy orientation rules. `-lattice_z N Z` uses explicit rank-1 generator Z; `-lattice N` normalizes/selects the generator internally.

`-euler_quad Nb Ng`. Uses high-order quadrature in $\cos\beta$ and periodic gamma points. It is slower than Sobol for broad scans but useful as a deterministic quadrature reference.

`-adaptive EPS`. Starts from an oldauto-based estimate, evaluates control quantities, and increases Sobol orientation count until relative changes are below EPS or the oldauto-derived cap is reached. The control includes the angular integral and selected M_{11} rows, especially near backscatter.

`-auto EPS`. Convenience mode: implies adaptive theta grid, auto phi grid, and adaptive Sobol orientations. It does not use a final regular oldauto grid.

`-autofull EPS`. Sequentially tunes:

$$n \;\to\; N_\varphi \;\to\; N_{\text{orient}}.$$

It is intended to search parameters, not to be the fastest production mode. It writes maps such as `_autofull_phi_by_theta.dat` and `_autofull_orient_by_theta.dat`.

`-oldautofull EPS`. Uses the autofull search but finishes with a regular oldauto-style $\beta \times \gamma$ grid. This was added because irregular Sobol/adaptive orientation sets interact poorly with mirror-sector reuse and some multigrid comparisons. The final orientation count is selected by `AutoFullOldautoEstimateForTa` and is at least an oldauto div4 grid in strict mode.

`-owen_avg K`, `-owen_seeds S...`. Only affects `-autofull` final averaging. It repeats the final Sobol average over K nested Owen seeds or over the explicit seed list.

5.7 Symmetry and mirror gamma

Particle classes provide a symmetry domain:

$$\beta \in [0, \beta_{\text{sym}}], \qquad \gamma \in [0, \gamma_{\text{sym}}].$$

`-sym Sb Sg` overrides this as:

$$\beta_{\text{sym}} = \pi / S_b, \qquad \gamma_{\text{sym}} = 2\pi / S_g.$$

`-mirror_gamma` halves the gamma domain:

$$\gamma_{\text{range}} = \gamma_{\text{sym}} / 2.$$

If it controls a regular gamma grid, the code rounds the gamma count to an even number. If `-mirror_gamma` or `-fft` is active, N_φ is rounded upward to a multiple of 6. This is required for exact hexagonal phi-sector mapping; direct non-FFT calculations merely warn when $N_\varphi \bmod 6 \neq 0$.

5.8 Scattering theta/phi grid flags

`-grid T1 T2 Nphi Ntheta`. Uniform full theta interval in degrees. The fourth argument is the number of theta intervals, so output has $N_\theta + 1$ rows. Example `-grid 0 180 600 180` writes 181 theta rows.

`-grid R Nphi Ntheta`. Backscatter cone grid from $\theta = \pi - R$ to $\theta = \pi$.

`-tgrid FILE`. Reads non-uniform theta values in degrees, one per line. These values become the exact output rows and the integration weights are midpoint-cell weights.

`-auto_tgrid EPS`. Performs recursive bisection using pilot M_{11} values. For an interval (θ_a, θ_b) it computes the midpoint and compares it with linear interpolation:

$$e = \frac{|M_{11}(\theta_m) - M_{11}^{lin}(\theta_m)|}{\max(1, |M_{11}(\theta_m)|, |M_{11}^{lin}(\theta_m)|)}.$$

If $e >$ `EPS` and the maximum depth/step limits allow it, the interval is split. The final full run uses the resulting non-uniform theta grid.

`-auto_phi, -nphi`. `-nphi` has highest priority. `-auto_phi` chooses an internally estimated value and then applies the multiple-of-6 rule when needed.

5.9 CUDA backend

`-gpu` moves Phase 2 diffraction to CUDA. The CPU still performs: particle rotations, shadow construction, geometric tracing, beam splitting, beam filtering, and prepared-beam packing. CUDA processes prepared beams over the output angular grid. The GPU build has two precision variants:

- `gpu/bin/mbs_po_gpu_float_fast`: single-precision CUDA diffraction; fastest on consumer GPUs. The near-singularity epsilon is clamped to at least 3×10^{-5} in `MBS_GPU_FLOAT`.
- `gpu/bin/mbs_po_gpu_double_fast`: double-precision CUDA diffraction; slower on consumer GPUs, often competitive on V100-class cards.

If CUDA errors occur the default behavior is fatal. Set `MBS_GPU_ALLOW_FALLBACK=1` only for debugging, because then a failed GPU calculation can silently continue on CPU.

5.10 FFT angular interpolation

`-fft` is an angular Fourier interpolation in output azimuth φ, not a spatial pFFT/FMM. The code computes diffraction directly on a reduced uniform phi grid of size

$$N_{\varphi,low} = N_{\varphi,full}/F,$$

transforms rows by cuFFT, zero-pads Fourier modes, inverse-transforms to $N_{\varphi,full}$, and adds the interpolated Mueller arrays to the final accumulator.

Default factor selection in `choose_fft_phi_factor()`:

$$F = \begin{cases} 8, & N_{\varphi,full} \geq 2400, \\ 6, & N_{\varphi,full} \geq 600, \\ 3, & N_{\varphi,full} \geq 240, \\ 2, & \text{otherwise.} \end{cases}$$

`MBS_FFT_PHI_FACTOR=N` overrides this. `MBS_FFT_THETA_FACTOR` can enable experimental theta interpolation for uniform compatible grids.

Quality/debug environment variables:

- `MBS_FFT_CHECK=1`: also computes a denser/direct reference and prints max/RMS relative errors. This is a diagnostic and costs time.
- `MBS_FFT_ADAPTIVE_PHI=1`: detects suspicious theta rows after FFT interpolation and recomputes selected rows directly.
- `MBS_FFT_ADAPTIVE_PHI_THRESHOLD`: default 0.04.
- `MBS_FFT_ADAPTIVE_PHI_MAX_ROWS`: default 8.
- `MBS_GPU_FFT_PAIR=1`: interpolates full and no-shadow arrays as a pair to reduce duplicated FFT setup.

5.11 Beam and trace cutoffs

Beam cutoff happens after tracing, before diffraction. For one orientation the code computes:

$$J_{\max} = \max_b \|J_b\|^2, \qquad A_{\max} = \max_b A_b, \qquad I_{\max} = \max_b(\|J_b\|^2 A_b).$$

A prepared beam is skipped if any enabled test is true:

$$\|J_b\|^2 < \epsilon_J J_{\max}, \quad A_b < \epsilon_A A_{\max}, \quad \|J_b\|^2 A_b < \epsilon_I I_{\max}.$$

Flags:

- `-beam_cutoff EPS`: shorthand for the J and area thresholds unless the separate flags override them.
- `-beam_cutoff_j EPS`: sets ϵ_J.
- `-beam_cutoff_area EPS`: sets ϵ_A.
- `-beam_cutoff_importance EPS`: sets ϵ_I.

Trace cutoff happens earlier while expanding internal beams. It has analogous flags `-trace_cutoff_j`, `-trace_cutoff_area`, and `-trace_cutoff_importance`; `-trace_cutoff EPS` sets the J and area tests. `-trace_max_beams N` aborts expansion for an orientation after N traced beam nodes. Cutoffs are speed/accuracy tradeoffs; for strict reference runs use zero cutoffs.

5.12 Multigrid and multikeq

`-multigrid Dmin Dmax N` generates N logarithmically spaced $D_{\max}$ values. `-multikeq Kmin Kmax N` generates logarithmically spaced k_{eq} values. `-multikeq_list FILE` uses exact values from the file.

With `-oldauto`, the particle is first resized to the largest requested size. The largest size determines the oldauto orientation grid and the shared beam cache. Smaller sizes reuse the traced

topology and rescale diffraction phases. This is fast, but for non-convex particles it assumes that the reference-size beam topology is acceptable for all smaller sizes. For maximum robustness use separate single-size runs or `-multigrid_parallel`.

`-multigrid_parallel J` starts up to J child processes. In GPU workflows this is useful for filling one GPU with multiple independent small jobs, but two heavy jobs on one GPU can reduce throughput. Use `-multigrid_threads T` to set OpenMP threads per child.

5.13 Checkpointing

`-checkpoint` is opt-in. For `-orientfile` it writes `_checkpoint.bin` after each chunk. For `-oldauto`/`-random` it writes `_oldauto_checkpoint.bin` after each completed beta ring. The checkpoint stores accumulated Mueller arrays, incoming energy, outgoing energy, OT extinction, flags, and a parameter hash. On restart with the same `-o` path and matching hash the run resumes from the next beta ring. Oldauto/random checkpointing is disabled under MPI.

5.14 Threading, MPI, and scheduling

`-threads N` sets OpenMP worker threads. By default the program sets OpenMP to physical cores, not hyperthreads. In `-oldauto -gpu` runs, CPU threads trace gamma chunks while the GPU diffracts prepared chunks. The log line

```
Oldauto/random memory: gamma chunk=8/303 per beta
```

means that only 8 gamma orientations are prepared at a time for that beta ring. `-chunk N` overrides this chunk size. MPI, when enabled in the CPU binary, splits beta rings across ranks for oldauto/random and reduces Mueller arrays and energies at the end.

5.15 All command-line flags

Release binaries intentionally show a shortened production-safe `-help`. The full list below is available with `-help-debug` or from binaries built with `make -C cpu debug`, `make -C gpu float_debug`, or `make -C gpu double_debug`. Hidden flags are still parsed in release binaries for backward compatibility with old scripts.

Flag	Current effect
`-p TYPE ...`	Built-in particle: hex, bullet, rosette, droxtal, concave hex, aggregate.
`-pf FILE`	Load particle geometry from file.
`-rs SIZE`	Resize file particle to $D_{\max} =$ `SIZE`.
`-k_eq K`	Resize file particle to $2\pi r_{\mathrm{eq}}/\lambda = K$.
`-ri Re Im`	Refractive index; nonzero Im enables absorption accounting.
`-w LAMBDA`	Wavelength in micrometers.
`-n N`	Maximum internal reflection/refraction depth.
`-po`, `-go`	Physical optics or geometrical optics mode.
`-fixed B G`	Single orientation in degrees.
`-random Nb Ng`	Regular beta/gamma grid.
`-oldauto DIV`	Physics-based regular grid from diffraction step, divided by DIV.
`-ring_points N`	Points per diffraction ring in oldauto estimates; default 3.
`-sobol N`	Sobol low-discrepancy orientations.
`-sobol_seed N S`	Sobol with explicit nested Owen seed.
`-sobol_ring Nb Ng`	Sobol beta with shifted uniform gamma ring.
`-hammersley N`	Hammersley orientations.
`-lattice N`, `-lattice_z N Z`	Rank-1 lattice orientations.

Flag	Current effect
`-euler_quad Nb Ng`	Gauss/periodic Euler quadrature.
`-montecarlo N`	Random Monte Carlo orientations.
`-orientfile FILE`	Read beta/gamma orientations from file.
`-b B1 B2, -g G1 G2`	Override beta/gamma ranges in degrees.
`-sym Sb Sg`	Override symmetry domain: π/S_b, $2\pi/S_g$.
`-mirror_gamma`	Halve gamma domain and enforce compatible counts.
`-adaptive EPS`	Adaptive Sobol orientation convergence.
`-auto EPS`	Auto theta, auto phi, adaptive orientations.
`-autofull EPS`	Search n, N_φ, and orientations.
`-oldautofull EPS`	Autofull search followed by regular oldauto final grid.
`-owen_avg K, -owen_seeds ...`	Repeat autofull final Sobol averaging over Owen seeds.
`-maxorient N`	Cap adaptive orientation count.
`-grid T1 T2 Nphi Ntheta`	Uniform output theta range and phi count.
`-grid R Nphi Ntheta`	Backscatter cone output grid.
`-tgrid FILE`	Explicit non-uniform theta grid.
`-auto_tgrid EPS`	Adaptive theta bisection by M_{11} interpolation error.
`-auto_phi`	Automatic N_φ estimate.
`-nphi N`	Highest-priority phi count override.
`-threads N`	OpenMP threads.
`-gpu, -cpu`	CUDA diffraction or forced CPU in GPU build.
`-fft`	cuFFT angular phi interpolation.
`-beam_cutoff*`	Post-trace beam skip thresholds.
`-trace_cutoff*`	During-trace beam expansion thresholds.
`-trace_max_beams N`	Max traced beam nodes per orientation.
`-gpu_trace`	Experimental CUDA prefilter for non-convex tracing candidates.
`-multigrid, -multikeq, -multikeq_list`	Multi-size scans.
`-multigrid_parallel J, -multigrid_threads T`	Independent child-process size parallelism.
`-checkpoint`	Save/resume long orientfile and oldauto/random runs.
`-save_betas`	Write per-beta and cumulative beta Mueller files.
`-full_only`	Default: only full output.
`-noshadow_output`	Also compute/write no-shadow output.
`-shadow_off`	Disable shadow beam; debug only.
`-incoh, -coh_orient`	Incoherent per-beam or legacy coherent-across-orientations mode.
`-abs, -abs_points N|all`	Force absorption and choose path sampling points.
`-ot_phase_shift F, -ot_phase_avg`	Diagnostic OT far-reference phase controls.
`-karczewski`	Use Karczewski polarization matrix option.
`-legacy_sign`	Restore old Fresnel sign convention.
`-pole`	Fast pole gamma shortcut.
`-forced_convex, -forced_nonconvex`	Force tracing branch.
`-r RATIO`	Legacy small-beam restriction ratio.
`-filter DEG, -point`	Legacy/backscatter restricted output options.
`-tr FILE, -all, -gr`	Trajectory-group diagnostics.
`-o NAME, -close, -log SEC`	Output path, exit after run, progress interval.
`-jones`	Write Jones matrices where supported.

6 Build

6.1 Intel / AVX-512

```
bash build.sh            # -> bin/mbs_po
```

`-O3 -march=native -mavx512f -mavx512dq -fopenmp`. Requires GCC ≥ 9.

6.2 AMD EPYC 7H12 (Zen 2, SP3)

```
bash build_epyc.sh       # -> bin/mbs_po_epyc
bash build_epyc_clang.sh # -> bin/mbs_po_epyc_clang
```

`-march=znver2`. No AVX-512; `fast_sincos_8x` falls back to 2×`fast_sincos_4x`.

NUMA: `OMP_PROC_BIND=close OMP_PLACES=cores OMP_NUM_THREADS=64`

6.3 AMD Zen 4 (Ryzen 7000, EPYC Genoa)

```
bash build_zen4.sh       # -> bin/mbs_po_zen4
```

`-march=znver4 -mavx512f -mavx512dq -mavx512vl`.

6.4 Generic AVX2

Remove `-mavx512*` from `build.sh`. Code auto-detects via `#ifdef __AVX512F__`.

7 Performance

7.1 Single-thread benchmarks

Hex D=H=10 μm, 128 Sobol, 48 ϕ × 181 θ, n=6:

Version	Phase 2	vs Original
Original (before optimizations)	∼320 s	1×
Optimized + batched sincos	4.6 s	∼**70**×
EPYC build (AVX2 only)	4.7 s	∼68×

7.2 Key optimizations

1. **PrecomputeEdgeData**: slopes, intercepts computed once per beam (not per direction).
2. **Theta-coefficients**: $A(\theta) = \sin\theta \cdot a_s + \cos\theta \cdot a_c + a_0$ (2 FMA).
3. **Batched sincos**: all vertex phases for all θ pre-computed before the θ-loop. AVX-512 `fast_sincos_8x` or AVX2 2×`fast_sincos_4x`.
4. **Polynomial sincos**: Cody–Waite range reduction + Chebyshev polynomial (∼14 decimal digits).
5. **Inline hot loop**: no heap allocation, stack-only `complex` arithmetic.
6. **OpenMP**: thread-local Mueller accumulators, `schedule(dynamic,1)`.
7. **Sobol + symmetry**: native C++ Sobol generator with particle symmetry reduction.

7.3 OpenMP scaling

Threads	Phase 2	Speedup
1	4.6 s	1.0×
4	1.7 s	2.7×
12 (SMT)	∼0.9 s	∼5×
64 (EPYC)	∼0.2 s*	∼23×*

*Estimated; use $\geq$512 orientations for 64 cores.

8 CUDA / FFT Recommendations for File Particles

For large file particles and shared size sweeps, use the CUDA diffraction backend with FFT phi interpolation and the global orientation scheduler:

```
MBS_SHARED_PIPELINE=1 MBS_SHARED_ORIENT_CHUNK=64 \
./bin/mbs_po_float_fast --pf Afine30.dat --multikeq_list keq_list.txt \
    --ri 1.6 0.002 -n 14 --po --oldauto 2 \
    --beam_cutoff_j 0.001 --beam_cutoff_area 0.002 \
    --trace_cutoff_importance 0.0001 --trace_max_beams 20000 \
    --grid 0 180 600 181 -w 1.064 --gpu --fft --close
```

8.1 `-gpu -fft`

`-gpu` moves the Phase 2 diffraction integral to CUDA. The CPU still performs ray tracing and prepares beam data. `-fft` enables angular Fourier interpolation in φ: diffraction is evaluated on a reduced direct azimuth grid and reconstructed to the requested N_φ by cuFFT. This is not aperture pFFT/FMM and not a 3D spatial FFT.

Recommended scattering grid for ADDA comparisons used here:

```
--grid 0 180 600 181
```

This gives 600 azimuth points and 181 theta points, including 0 and 180 degrees.

8.2 `MBS_SHARED_ORIENT_CHUNK=N`

Shared `-oldauto -multikeq*` GPU runs use a global orientation scheduler. Orientations are indexed as

$$i = i_\beta N_\gamma + i_\gamma.$$

The scheduler traces contiguous chunks of `N` orientations and immediately sends each completed chunk to CUDA diffraction. This avoids waiting for a whole beta block when one non-convex orientation is slow.

Default for GPU shared multikeq:

```
MBS_SHARED_ORIENT_CHUNK=64
```

Memory scales approximately as

$$M_{\text{host}} \sim N_{\text{chunk}} \langle N_{\text{beams/orientation}} \rangle,$$

instead of

$$M_{\text{host}} \sim N_{\text{betaGroup}} N_\gamma \langle N_{\text{beams/orientation}} \rangle.$$

Practical choices:

- `MBS_SHARED_ORIENT_CHUNK=64`: recommended default for Afine30 / Greek file-particle runs.
- `MBS_SHARED_ORIENT_CHUNK=32`: lower latency; use when GPU utilization has long zero-utilization gaps.
- `MBS_SHARED_ORIENT_CHUNK=128`: fewer CUDA launches; test when GPU launch overhead dominates and host RAM is available.

On Afine30 with `-grid 0 180 600 181 -gpu -fft -oldauto 2`, chunks of 32–64 improved the observed scheduler rate from about 4.3 orientations/s to about 5.5 orientations/s on the RTX 4070 SUPER node and reduced host RAM from roughly 10 GB to below 2 GB.

8.3 MBS_SHARED_PIPELINE=1

This overlaps CPU tracing of chunk $k+1$ with CUDA diffraction of chunk k. Use it with the global scheduler:

```
MBS_SHARED_PIPELINE=1 MBS_SHARED_ORIENT_CHUNK=64
```

If the GPU is still idle for long intervals, reduce the chunk to 32. If the GPU is busy but many tiny launches dominate, increase it to 128.

8.4 MBS_SHARED_BETA_GROUP=N

This is the older beta-block batching control. Large values reduce CUDA launch count but can be slower on non-convex particles because one hard orientation holds the whole beta group. With the global scheduler enabled, treat this as a diagnostic/legacy knob. Use:

```
MBS_SHARED_BETA_GROUP=1  # lowest latency
MBS_SHARED_BETA_GROUP=2  # conservative beta-block test
MBS_SHARED_BETA_GROUP=4  # larger packet, more RAM, longer tails
```

Recommended production mode is the global orientation chunk, not large beta groups.

9 Bugfixes

9.1 Forward-direction Fresnel sign (2026-03-22)

Bug: sign error in the forward-direction special case ($A{\approx}0, B{\approx}0$) of the Kirchhoff integral. The edge-sum converges to $-F_{\text{inv}} \cdot \text{area}$ but the shortcut used $+F_{\text{inv}} \cdot \text{area}$.

Symptom: $M_{11}(0°)$ anomalously small in coherent mode for small particles. Example (hex $D{=}H{=}3\,\mu$m): $M_{11}(0)/M_{11}(0.1°) = 0.019$ (should be ≈ 1.03).

Fix: negate the forward-direction formula in 6 locations:

- `HandlerPO.cpp`: lines 970, 1333, 1889
- `Handler.cpp`: lines 255, 473, 642

Incoherent mode unaffected ($|J|^2$ is sign-independent). Q_{sca} changed by $\sim$0.06%. See `BUGFIX_forward_direct` for full details.

10 Known Limitations

1. $Q_{\text{sca}} > 2$ **at large** x: PO does not enforce the optical theorem.
2. M_{33}, M_{34}, M_{44}: RotateJones vs Karczewski give different off-diagonal elements (same M_{11}).
3. **Small particles** ($x < 20$): PO not valid. Use ADDA or T-matrix.
4. **Backscattering convergence**: $M_{11}(180°)$ needs 10–100$\times$ more orientations.
5. **LTO**: `-flto` causes $\sim$20% regression. Do not use.

11 File Listing

File	Description
`build.sh`	Intel/AVX-512 build
`build_epyc.sh`	AMD EPYC 7H12 (Zen 2) build
`build_epyc_clang.sh`	EPYC + Clang/AOCC build
`build_zen4.sh`	AMD Zen 4 (Ryzen 7000/Genoa) build
`Makefile`	Legacy Makefile (Intel)
`Makefile.epyc`	Makefile for EPYC
`tests/run_tests.sh`	Regression test suite
`MANUAL.md`	This manual (Markdown source)
`BUGFIX_forward_direction_sign.md`	Bugfix documentation
`docs/bugfix_15x_mismatch.pdf`	15× mismatch investigation

12 Bugfix: 15× Mismatch (old vs new binary)

Comparison of `-fixed 45 30` between old and new binaries showed a 15.5× difference in M_{11}.

12.1 Root causes

1. **Old `-fixed` never rotated the particle.** `ScatteringConvex::ScatterLight()` has `m_particle->Rotate` commented out, and old `TracerPO::TraceFixed` did not call it either. Result: `-fixed 45 30` always gave the default orientation. New code correctly adds `m_particle->Rotate(b, g, 0)` in `TraceFixed`.
2. **Uninitialized `d_param` in particle constructors.** `Point3f()` does not zero `coordinates[3]`. No constructor called `SetDParams()` after `Reset()`. Fixed: `SetDParams()` added after `Reset()` in all 8 particle constructors.
3. **Mixed `fast_exp_im`/`exp_im` in fallback paths.** `DiffractIncline`, `DiffractInclineFast`, `ComputeFnJones`, and `ApplyDiffraction*` had inconsistent precision. Fixed: all fallback paths use `exp_im` (glibc sincos).

12.2 Validation

Test	Max diff	Mode	Notes
`-fixed 0 0 -incoh`	exact	incoh	per-beam $\|J\|^2$ identical
`-random 64 43 -incoh`	exact	incoh	per-beam physics identical
`-random 32 22 -coh_orient`	10^{-7}	coh	machine precision
`-random 64 43` (default)	0.007%	incoh avg	coh vs incoh averaging
HandleBeams inline vs fallback	identical		SIMD optimizations verified

Speedup: 55× (23 min → 25 sec at 2860 orientations, 4 threads).

d_param note: `d_param` is the plane equation parameter ($ax + by + cz + d = 0$) stored in `Point3f::coordinates[3]`. It was uninitialized before `Rotate()` call. `SetDParams()` now added to all particle constructors. This changes coherent single-orientation M_{11} by ∼7% (phase-sensitive), but incoherent $|J|^2$ is identical. After orientation averaging the effect is $< 0.01\%$.

Impact: Only `-fixed` mode affected. Orientation averaging modes (`-random`, `-sobol`, `-adaptive`, `-auto`) always call `Rotate()` via `TracerPOTotal`, so d_param is always initialized correctly.

See `docs/bugfix_15x_mismatch.pdf` and `tests/reference_test/` for full reports and plots.

13 New Features (March 2026)

13.1 Adaptive theta grid (`-auto_tgrid EPS`)

Replaces the static zone-based theta grid with recursive bisection. For each interval $[\theta_i, \theta_{i+1}]$: computes M_{11} at the midpoint, compares with linear interpolation. If relative error > EPS, splits the interval.

- Minimum step: $\lambda/(8D)$ (quarter-Nyquist)
- Maximum depth: 8 levels
- Probe orientations: div16 physics estimate (same as adaptive start)
- Result: 2–3× fewer theta points than static grid

Usage: `-auto_tgrid 0.05` (5% tolerance). Works with `-sobol`, `-random`, `-auto`.

13.2 Flag priorities

Multiple flags can set the same parameter. Resolution order (highest priority first):

Theta grid (N_θ and theta values)

1. `-tgrid FILE` — explicit non-uniform grid from file
2. `-grid T1 T2 Nphi Ntheta` — uniform grid from command line
3. `-auto_tgrid EPS` — adaptive bisection (probes M_{11})
4. `-auto` — implies `-auto_tgrid 0.05` (bisection)
5. `-oldauto DIV` — static zone grid from `ApplyAutoThetaGrid`
6. Default: placeholder (1 theta point)

Phi grid (N_φ)

1. `-nphi N` — explicit override (highest priority)
2. `-grid T1 T2 Nphi Ntheta` — `Nphi` is the 3rd argument
3. `-auto_phi` — formula $N_\varphi = \min(360,\ x/5 + 48)$
4. `-auto` — implies `-auto_phi`
5. `-oldauto DIV` — same formula as `-auto_phi`
6. Default: 1 (useless without override)

Orientation method

Mutually exclusive (first matched wins):

1. `-fixed BETA GAMMA` — single orientation
2. `-oldauto DIV` — physics-based $\beta \times \gamma$ grid
3. `-random Nb Ng` — regular grid
4. `-montecarlo N` — random uniform
5. `-orientfile FILE` — from file
6. `-auto EPS` / `-autofull EPS` — adaptive Sobol
7. `-adaptive EPS` — adaptive Sobol (manual grid)
8. `-sobol N` — fixed-count Sobol

Beam cutoff

1. `-beam_cutoff EPS` — explicit threshold
2. `-auto`/`-adaptive` — uses their EPS as cutoff
3. Default: 0 (no cutoff)

Coherent/incoherent

1. Default: coherent (Jones summed across beams per orientation)
2. `-incoh`: incoherent (Mueller per beam)
3. `-coh_orient`: coherent across ALL orientations (legacy)

Note: `-grid T1 T2 Nphi Ntheta` — the 3rd value is N_φ (not N_θ!). This is the original MBS convention. Use `-nphi` to avoid confusion.

13.3 Multi-size (`-multigrid Dmin Dmax N`)

Generates N sizes from $D_{\min}$ to $D_{\max}$ in logarithmic scale. For each size: rescales particle via `Resize()`, runs full `TraceFromSobol` (same optimized path as standalone). Results bit-exact with separate runs.

```
# 10 sizes from D=50 to D=500, reference particle D=500
bin/mbs_po --po --sobol 1024 --multigrid 50 500 10 \
    -p 1 500 62.5 -w 0.532 --ri 1.31 0 -n 8 \
    --grid 0 180 360 180 --close -o results
```

Output: `results_x{N}.dat` for each size parameter $x = \pi D/\lambda$.

13.4 Per-beta save (`-save_betas`)

With `-random`: writes per-beta Mueller contribution and cumulative Mueller to `_betas/` subfolder. Useful for convergence analysis and crash recovery.

13.5 Parallel Phase 1 (tracing)

Beam tracing (Phase 1) is now parallelized with OpenMP. Each thread gets its own copy of `Particle` + `Scattering` via `CloneFor()`. Phase 1 speedup: 5.4× on 12 threads.

13.6 Additional bugfixes

- `-montecarlo` missing handler config (isCoh, m_wave, shadowOff)
- `DiffractControlPoints`: wrong M11 formula (missing d01, d10 terms)
- `-karczewski` ignored in fast diffraction path
- `rdtsc` inline asm incorrect on x86-64
- `malloc` leak in `Particle::SetFromFile`
- OOM in `TraceAdaptive`: sub-chunked to 4096 max
- `-grid` conflicts with auto theta/phi resolved
- `-oldauto` N rounding: ceil instead of ODD (smoother size scans)
- `-point` crash from uninitialized handler pointer
- MPI missing `}` in `TraceAdaptive` convergence block

14 Performance: MBS-fast vs MBS-raw (original)

All benchmarks: hex column $L = 199.5$, $D = 24.9375$ µm, $\lambda = 0.532$, $m = 1.31$.

The original MBS-raw code is **single-threaded**, uses standard `sin`/`cos` (glibc), computes the diffraction integral with full `exp_im` per edge, and has no beam filtering or adaptive grids.

14.1 Architecture comparison

Component	MBS-raw (original)	MBS-fast
Phase 1 (tracing)	1 thread, sequential	OpenMP, `CloneFor()` per thread
Phase 2 (diffraction)	1 thread, `exp_im` per edge	OpenMP, batched `fast_sincos` (AVX)
Edge integral	`ChangeCoordinateSystem` per vertex	Precomputed edges + slopes
A,B coefficients	Recomputed per (beam, θ, φ)	Theta-coefficients: hoist per φ
Vertex phases	$2 \times$ `exp_im` per edge per θ	$1 \times$ `fast_sincos` per vertex, batched
Beam filtering	None (all beams computed)	Two-threshold cutoff (skip 45–80%)
θ grid	Uniform or hand-tuned zones	Adaptive bisection (`-auto_tgrid`)
φ grid	Manual `-grid`	Auto: $N_\varphi = x/5 + 48$
Orientations	Regular $\beta \times \gamma$ grid	Sobol quasi-random + adaptive
Memory	Full arrays, no chunking	Chunked streaming (4096 max)

14.2 Speedup breakdown

Each optimization contributes a multiplicative factor. Combined effect depends on which phase dominates (Phase 2 for most cases).

Optimization	Where	Factor	Notes
Phase 2 (diffraction) — typically 90–95% of total time:			
OpenMP (thread-local Mueller)	Phase 2	$N_{\text{cores}}\times$	12 cores → 10–11×
Batched sincos (AVX-512 8-wide)	Phase 2	∼2×	polynomial approx, SIMD
Precomputed edge data	Phase 2	∼1.5×	slopes, intercepts cached
Theta-coefficients	Phase 2	∼1.3×	A,B decomposed per φ
Beam cutoff	Phase 1+2	∼2–4×	skip weak beams
Adaptive theta (bisection)	Grid	∼2–3×	fewer θ points
Phase 1 (tracing) — typically 5–10% of total time:			
OpenMP (`CloneFor`)	Phase 1	$N_{\text{cores}}\times$	12 cores → 5.4×
Adaptive mode:			
DiffractControlPoints	Convergence	∼100×	4 angles, not full grid
Incremental Sobol	Convergence	∼3.5×	reuse previous orientations
Physics-based start (div16)	Convergence	∼2×	skip initial low-N iterations

Combined estimate for 12 cores:

Phase 2: 10× (OMP) × 2× (AVX) × 1.5× (edges) × 1.3× (theta-coeff) × 2× (beam cutoff) ≈ **78**×. With adaptive theta (2×): ≈ **156**×.

14.3 Measured speedup

Test	Params	Old (1 thr)	New	Speedup	Accuracy
Forward 0–25°	$n=4$, 64×43, 4 thr	17 min	16 sec	**63**×	0.007%
Forward 0–25°	$n=4$, 64×43, 12 thr	17 min	11.5 sec	**89**×	0.007%
Backward 160–180°	$n=11$, 64×43, 4 thr	47 min	27 sec	**105**×	0.8%
Backward 160–180°	$n=11$, 64×43, 12 thr	47 min	17 sec	**168**×	0.8%
`-coh_orient`	32×22, 4 thr	30 sec	0.5 sec	**60**×	10^{-7}
Phase 1 only	$n=8$, 8192, 12 thr	5.2 sec	0.96 sec	**5.4**×	exact

On 128-core EPYC cluster with MPI: ∼1000×+ expected.

14.4 Accuracy

- `-coh_orient`: machine precision (10^{-7}) match with original

- `-incoh`: exact per-beam $|J|^2$ — physics identical
- Default mode: $< 0.01\%$ at convergence (coh vs incoh averaging)
- Forward peak ($\theta = 0$): 0.39% from `d_param` fix (correct value)
- All optimizations verified: inline path = fallback path (bit-exact)